

Apprendre Kubernetes avec nginx

Tous les concepts essentiels expliqués pas à pas, illustrés avec une seule image : nginx

Formation DevOps • avec illustrations

Formateur : Haithem Mihoubi

10 modules • Pods, Deployments, Services, ConfigMaps, Volumes, Ingress • 2026

Table des matières

00	Introduction : pourquoi Kubernetes ?	3
01	L'architecture d'un cluster Kubernetes	6
02	Les Pods : la plus petite unité	9
03	Deployments & ReplicaSets : la résilience	12
04	Mises à jour sans coupure & rollback	15
05	Les Services : exposer nginx	18
06	ConfigMap & Secret : configurer nginx	22
07	Le stockage persistant	26
08	L'Ingress : router le trafic HTTP	30
09	Pratique, santé & checklist	34

Introduction : pourquoi Kubernetes ?

1. Le problème : faire tourner des conteneurs « en vrai »

Avec Docker, on sait lancer un conteneur nginx :

```
docker run -d -p 80:80 nginx
```

Parfait sur **une** machine, pour **un** conteneur. Mais en production, on veut :

- **Plusieurs copies** (réplicas) pour tenir la charge.
- Qu'une copie **redémarre toute seule** si elle plante.
- **Répartir** ces copies sur plusieurs serveurs.
- Mettre à jour la version **sans coupure**.
- Une **adresse stable** alors que les conteneurs vont et viennent.
- Gérer **config, secrets, stockage, réseau...**

Faire tout cela à la main avec `docker run`, c'est ingérable. Il faut un **orchestrateur**.

2. La solution : Kubernetes (K8s)

Kubernetes est le système open source qui **orchestre** des conteneurs sur un ensemble de machines : il les déploie, les réplique, les surveille, les répare, les met à jour et les expose au réseau — **automatiquement**.

Le mot vient du grec *kubernêtês* (« timonier »). On l'abrège **K8s** (k + 8 lettres + s).

L'idée centrale : le modèle déclaratif

Avec Docker, on donne des **ordres** (« lance ce conteneur »). Avec Kubernetes, on décrit un **état désiré** (« je veux **3** nginx en permanence ») dans un fichier YAML. Kubernetes travaille **en boucle** pour que l'état réel corresponde toujours à l'état désiré.

Approche impérative (Docker)	Approche déclarative (Kubernetes)
« Lance un conteneur »	« Je veux qu'il y ait 3 nginx »
Si ça plante, c'est fini	Si ça plante, K8s le recrée
Vous gérez chaque étape	Vous décrivez la cible, K8s s'en charge

C'est la notion de **réconciliation** : un Pod meurt ? Kubernetes en relance un. Vous demandez 5 réplicas au lieu de 3 ? Il en crée 2 de plus. **Vous décrivez, il exécute.**

3. Notre fil rouge : nginx

Tout au long de ce cours, **un seul exemple** sert à illustrer chaque concept : l'image `nginx` (un serveur web léger, public, qui démarre en une seconde). C'est l'idéal pour apprendre : on se concentre sur **Kubernetes**, pas sur l'application.

À la fin, vous saurez déployer nginx :

- en **plusieurs réplicas** auto-réparés (Pods, Deployment) ;
- avec une **adresse stable** et un accès externe (Service, Ingress) ;
- **configuré** sans rebuild (ConfigMap, Secret) ;
- avec du **stockage persistant** (PV, PVC) ;
- mis à jour **sans coupure** (rolling update) et avec **rollback**.

4. Le vocabulaire en une page

Objet	Rôle (avec nginx)
Pod	La plus petite unité : enveloppe le conteneur nginx
ReplicaSet	Maintient N copies identiques du Pod nginx
Deployment	Gère les ReplicaSets : mises à jour, rollback, scaling
Service	Adresse réseau stable devant les Pods nginx
Ingress	Routeur HTTP/HTTPS à l'entrée du cluster
ConfigMap / Secret	Configuration et données sensibles de nginx
PV / PVC	Stockage persistant monté dans le Pod
Namespace	Cloison logique pour organiser les objets

5. Comment essayer Kubernetes

Pas besoin d'un cloud pour apprendre : un cluster local suffit.

Outil	Pour
minikube	Cluster local à un node, simple
kind	Kubernetes « in Docker », rapide pour tester
k3s	Distribution légère, idéale petits serveurs
Docker Desktop	Active Kubernetes en une case à cocher

```
# Exemple avec minikube
minikube start
kubectl get nodes      # le cluster répond ?
```

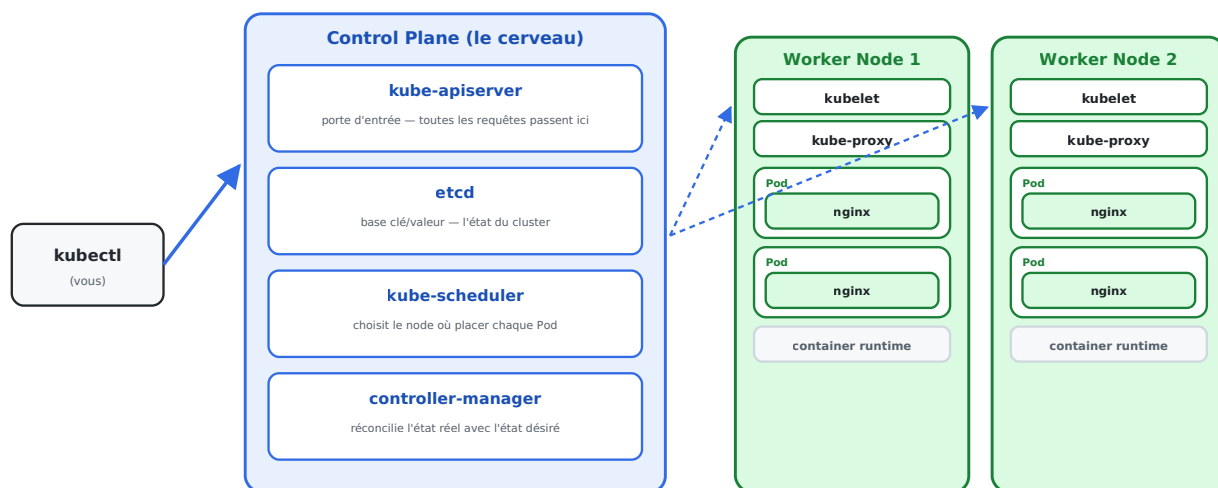
`kubectl` (prononcé « cube-cti-el ») est l'outil en ligne de commande pour **parler au cluster**. On l'utilisera dans tout le cours.

Pré-requis : connaître Docker (images, conteneurs, ports) — voir le cours « *Dockeriser ses applications* ». Aucune connaissance préalable de Kubernetes n'est requise.

L'architecture d'un cluster Kubernetes

Un **cluster** Kubernetes est un ensemble de machines (des *nodes*) qui travaillent ensemble. Elles se répartissent en deux rôles : **piloter** et **exécuter**.

Architecture d'un cluster Kubernetes



Le Control Plane décide ; les Worker Nodes exécutent les conteneurs. Tout dialogue passe par kube-apiserver.

Le Control Plane décide ; les Worker Nodes exécutent les conteneurs. Tout dialogue passe par kube-apiserver.

1. Les deux mondes : Control Plane & Worker Nodes

	Control Plane	Worker Nodes
Rôle	Le cerveau : décide et surveille	Les bras : font tourner les Pods
Contient	apiserver, etcd, scheduler, controllers	kubelet, kube-proxy, runtime, Pods
Analogie	La tour de contrôle	Les pistes et les avions

2. Les composants du Control Plane

kube-apiserver — la porte d'entrée

Tout passe par lui : `kubectll`, les autres composants, les outils. C'est l'unique point d'accès au cluster, qui valide et enregistre chaque requête. Quand vous tapez `kubectll apply`, vous parlez à l'apiserver.

etcd — la mémoire du cluster

Une base de données **clé/valeur** qui stocke **tout l'état** du cluster : quels objets existent, leur configuration, leur état désiré. Si etcd est perdu, le cluster est perdu — d'où l'importance de le **sauvegarder**.

kube-scheduler — le placeur

Quand un nouveau Pod doit tourner, le scheduler choisit **sur quel node** le placer, selon les ressources disponibles (CPU, mémoire), les contraintes et les affinités.

controller-manager — le gardien de l'état désiré

Il fait tourner les **boucles de réconciliation**. Exemple : le *ReplicaSet controller* vérifie en continu « y a-t-il bien 3 nginx ? ». S'il en manque un, il en commande un nouveau.

3. Les composants d'un Worker Node

kubelet — l'agent du node

Sur chaque worker, le kubelet reçoit les ordres de l'apiserver et **fait tourner les conteneurs** via le runtime. Il rapporte aussi l'état des Pods (vivants ? en bonne santé ?).

kube-proxy — le routeur réseau

Gère les règles réseau du node pour que les **Services** fonctionnent : il route le trafic vers le bon Pod, où qu'il soit.

container runtime — le moteur de conteneurs

Le logiciel qui exécute réellement les conteneurs (containerd, CRI-O...). C'est lui qui télécharge l'image `nginx` et lance le processus.

4. Le cycle de vie d'une demande

Quand vous lancez `kubectl apply -f nginx.yaml` pour créer 3 nginx :

1. **kubectl** envoie le YAML à **kube-apiserver**.
2. L'apiserver valide et écrit l'état désiré dans **etcd**.
3. Le **controller-manager** voit « 3 Pods voulus, 0 existant » → crée 3 Pods.
4. Le **scheduler** assigne chaque Pod à un **node**.
5. Le **kubelet** du node concerné demande au **runtime** de lancer le conteneur nginx.
6. **kube-proxy** met en place le routage réseau.

Le principe à retenir : personne ne « lance » directement un conteneur. On déclare un état dans etcd, et les **contrôleurs** convergent vers cet état. C'est ce qui rend Kubernetes auto-réparateur.

5. Quelques commandes pour explorer le cluster

```
kubectl get nodes           # lister les machines du cluster
kubectl get nodes -o wide   # avec IP, OS, version
kubectl cluster-info        # adresses du control plane
kubectl get pods -A         # tous les Pods, tous les namespaces
kubectl describe node <nom> # détail d'un node (ressources, Pods)
```

Dans la suite, on ne touchera plus directement à ces composants : on travaillera avec des **objets** (Pods, Deployments, Services...) que les composants se chargent de réaliser.

Les Pods : la plus petite unité

Le **Pod** est l'objet fondamental de Kubernetes. On ne déploie jamais un conteneur directement : on déploie un **Pod** qui **enveloppe** le conteneur.

Le Pod : la plus petite unité déployable

Un Pod enveloppe un (ou plusieurs) conteneurs qui partagent réseau et stockage



Règle : on ne lance jamais un conteneur seul — toujours dans un Pod. En général : 1 Pod = 1 conteneur applicatif.

Un Pod enveloppe un (ou plusieurs) conteneurs qui partagent une IP et des volumes.

1. Qu'est-ce qu'un Pod ?

- C'est une **enveloppe** autour d'un ou plusieurs conteneurs.
- Les conteneurs d'un même Pod partagent **la même IP** et **les mêmes volumes**, et communiquent via `localhost`.
- Un Pod tourne **toujours entièrement sur un seul node**.
- Un Pod est **éphémère** : on ne le répare pas, on le **remplace**. Il reçoit une nouvelle IP à chaque création.

En pratique : 1 Pod = 1 conteneur applicatif. Le cas multi-conteneurs existe (motif *sidecar* : un conteneur principal + un auxiliaire), mais reste l'exception.

2. Mon premier Pod nginx

Deux façons de créer un Pod nginx.

Méthode rapide (impérative)

```
kubectl run nginx --image=nginx:1.27 --port=80
kubectl get pods
```

Méthode recommandée (déclarative, en YAML)

Fichier `nginx-pod.yaml` :

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    app: nginx          # une étiquette, essentielle pour les Services
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      ports:
        - containerPort: 80
```

```
kubectl apply -f nginx-pod.yaml
```

Anatomie d'un manifeste

Tous les objets Kubernetes partagent les **quatre mêmes champs** :

Champ	Rôle
<code>apiVersion</code>	Version de l'API de l'objet (<code>v1</code> , <code>apps/v1</code> ...)
<code>kind</code>	Le type d'objet (<code>Pod</code> , <code>Deployment</code> , <code>Service</code> ...)
<code>metadata</code>	Nom, namespace, labels
<code>spec</code>	L'état désiré — le cœur de la définition

3. Inspecter et déboguer un Pod

```
kubectl get pods          # liste + statut (Running, Pending...)
kubectl get pods -o wide  # + IP du Pod et node
kubectl describe pod nginx # événements détaillés (très utile au débogage)
kubectl logs nginx        # logs du conteneur (ici, ceux de nginx)
kubectl exec -it nginx -- bash # ouvrir un shell DANS le conteneur
kubectl port-forward pod/nginx 8080:80 # accéder à nginx depuis le PC : localhost:8080
```

Le réflexe débogage : un Pod ne démarre pas ? `kubectl describe pod <nom>` montre les événements (image introuvable, ressources insuffisantes...). Puis `kubectl logs <nom>` pour les erreurs applicatives.

4. Le cycle de vie d'un Pod

Phase	Signification
Pending	Accepté, mais pas encore lancé (téléchargement de l'image, attente de place)
Running	Au moins un conteneur tourne
Succeeded	Tous les conteneurs ont terminé avec succès (tâches ponctuelles)
Failed	Au moins un conteneur s'est terminé en erreur
CrashLoopBackOff	Le conteneur plante puis redémarre en boucle → voir les logs

5. La grande limite du Pod « nu »

Si vous supprimez ce Pod, ou si son node tombe, **il disparaît définitivement** :

```
kubectl delete pod nginx
kubectl get pods          # plus rien – aucun remplacement automatique
```

C'est voulu : **on n'utilise presque jamais un Pod seul**. On confie leur gestion à un objet de plus haut niveau qui les **recrée** et les **réplique** : le **Deployment**. C'est l'objet du module suivant.

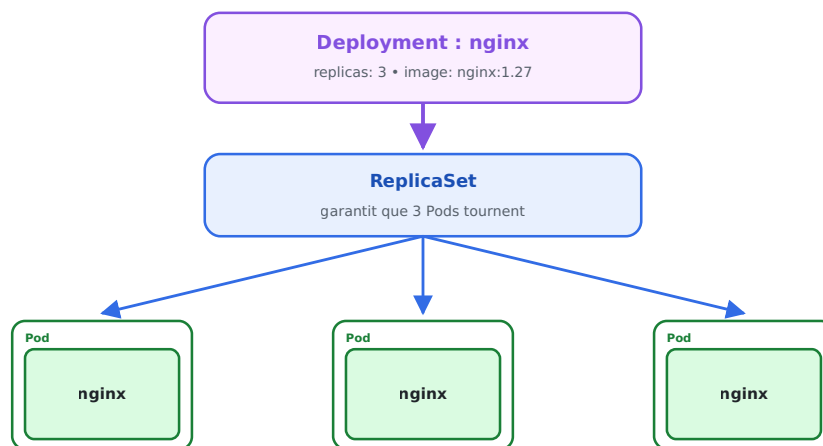
À retenir : le Pod est l'unité d'exécution, mais il est jetable. La résilience vient des contrôleurs (ReplicaSet, Deployment) qui veillent dessus.

Deployments & ReplicaSets : la résilience

Un Pod seul est fragile. Le **Deployment** est l'objet qu'on utilise **réellement** pour faire tourner une application : il garantit un nombre de copies et les répare tout seul.

Deployment > ReplicaSet > Pods

Vous déclarez « je veux 3 nginx » ; Kubernetes s'occupe d'y arriver et de le maintenir



Si un Pod meurt, le ReplicaSet en recrée un aussitôt : c'est l'auto-réparation (self-healing).

Vous déclarez « 3 nginx » ; le Deployment crée un ReplicaSet qui maintient 3 Pods en vie.

1. La hiérarchie : Deployment > ReplicaSet > Pods

Objet	Responsabilité
Deployment	Gère les versions et le scaling ; pilote le ReplicaSet
ReplicaSet	Maintient en permanence N copies identiques d'un Pod
Pod	Exécute le conteneur nginx

Vous interagissez avec le **Deployment** ; il crée et pilote les ReplicaSets, qui eux-mêmes créent les Pods. On ne manipule presque jamais le ReplicaSet directement.

2. Un Deployment nginx

Fichier `nginx-deployment.yaml` :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 3                # — l'état désiré : 3 Pods
  selector:
    matchLabels:
      app: nginx            # quels Pods ce Deployment gère-t-il ?
  template:                 # — le « moule » des Pods créés
    metadata:
      labels:
        app: nginx         # doit correspondre au selector ci-dessus
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
```

```
kubectl apply -f nginx-deployment.yaml
kubectl get deployments
kubectl get replicaset
kubectl get pods          # → 3 Pods nginx-xxxx
```

Le champ `template` est un mini-manifeste de Pod : c'est le **moule** à partir duquel chaque réplica est fabriqué. Le **`selector`** relie le Deployment aux Pods via leurs **labels**.

3. L'auto-réparation (self-healing)

C'est la magie du modèle déclaratif. Supprimez un Pod « à la main » :

```
kubectl delete pod nginx-7c5ddbdf54-abcde
kubectl get pods          # un NOUVEAU Pod apparaît aussitôt → toujours 3
```

Le ReplicaSet a constaté « 2 Pods alors qu'il en faut 3 » et en a **recréé un** immédiatement. Même chose si un node tombe : les Pods sont recréés ailleurs.

4. Le scaling : changer le nombre de réplicas

À la volée

```
kubectl scale deployment nginx --replicas=5
kubectl get pods          # → 5 Pods maintenant
```

Dans le YAML (recommandé, versionné)

On modifie `replicas: 5` puis `kubectl apply -f nginx-deployment.yaml`.

Scaling automatique (HPA)

Kubernetes peut **adapter le nombre de Pods à la charge CPU** :

```
kubectl autoscale deployment nginx --min=2 --max=10 --cpu-percent=70
```

→ entre 2 et 10 Pods selon la charge. C'est le *Horizontal Pod Autoscaler*.

5. Pourquoi toujours passer par un Deployment ?

Sans Deployment (Pod nu)	Avec Deployment
Pod supprimé = perdu	Pod recréé automatiquement
Pas de réplication	<code>replicas: N</code>
Pas de mise à jour propre	Rolling update + rollback (module suivant)
Pas de scaling	<code>kubectl scale</code> / HPA

6. Commandes utiles

```
kubectl get deployment nginx          # statut : READY 3/3
kubectl describe deployment nginx      # détails, événements, stratégie
kubectl get pods -l app=nginx          # filtrer par label
kubectl delete deployment nginx        # supprime le Deployment ET ses Pods
```

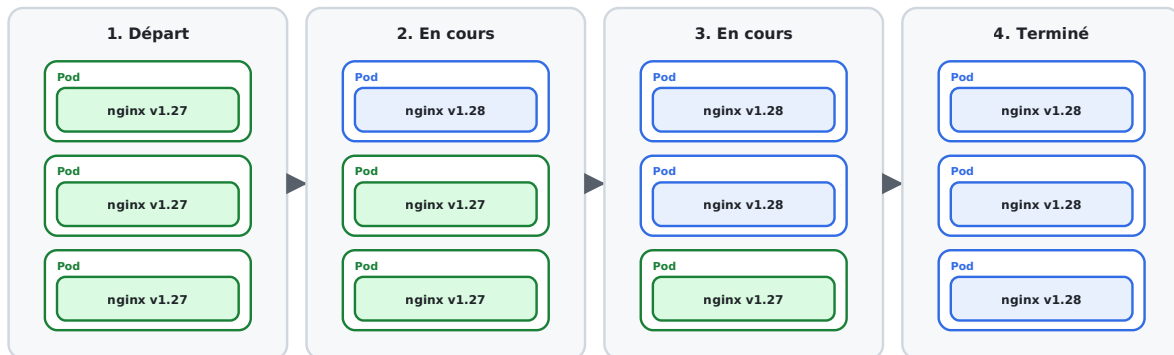
À retenir : le Deployment, c'est l'unité de déploiement standard. Il transforme des Pods jetables en service **résilient** et **scalable**. Le module suivant montre comment il met à jour nginx **sans aucune coupure**.

Mises à jour sans coupure & rollback

Mettre à jour nginx de `1.27` à `1.28` ne doit **jamais** couper le service. Le Deployment gère cela nativement grâce au **Rolling Update**.

La mise à jour progressive (Rolling Update)

`kubectl set image ... nginx:1.27 → nginx:1.28` — zéro coupure



Kubernetes remplace les Pods un par un : il en crée un neuf, attend qu'il soit prêt, puis supprime un ancien.

Kubernetes remplace les Pods un par un : il en crée un neuf, attend qu'il soit prêt, puis supprime un ancien.

1. Le principe du Rolling Update

Plutôt que tout arrêter puis tout relancer (coupure), Kubernetes remplace les Pods **progressivement** :

1. Il crée un Pod en nouvelle version (`1.28`).
2. Il **attend qu'il soit prêt** (voir *readiness probe*, module 09).
3. Il supprime un Pod en ancienne version (`1.27`).
4. Il répète jusqu'à ce que tous les Pods soient à jour.

À tout instant, il reste assez de Pods pour **servir le trafic : zéro coupure**.

2. Déclencher une mise à jour

En ligne de commande

```
kubectl set image deployment/nginx nginx=nginx:1.28
```

En YAML (recommandé)

On change `image: nginx:1.27` → `image: nginx:1.28`, puis :

```
kubectl apply -f nginx-deployment.yaml
```

Suivre le déroulement

```
kubectl rollout status deployment/nginx      # progression en direct
kubectl get pods -w                          # voir les Pods se remplacer (-w = watch)
```

3. Régler la stratégie de déploiement

On contrôle finement le rythme dans le `spec` du Deployment :

```
spec:
  replicas: 4
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1      # au plus 1 Pod indisponible pendant la mäj
      maxSurge: 1           # au plus 1 Pod supplémentaire créé temporairement
```

Paramètre	Effet
<code>maxUnavailable</code>	Combien de Pods peuvent manquer pendant la mäj (capacité minimale)
<code>maxSurge</code>	Combien de Pods en plus on s'autorise (vitesse de bascule)

Alternative Recreate : `strategy.type: Recreate` supprime tous les anciens Pods avant de créer les nouveaux. Il y a une **coupure**, mais c'est parfois nécessaire (ex. : incompatibilité entre deux versions qui ne peuvent pas coexister).

4. L'historique et le rollback

Kubernetes **mémore** les versions successives du Deployment.

```
kubectl rollout history deployment/nginx      # lister les révisions
kubectl rollout history deployment/nginx --revision=2  # détail d'une révision
```

Revenir en arrière en cas de problème

La nouvelle version est boguée ? On revient à la précédente **en une commande** :

```
kubectl rollout undo deployment/nginx      # revenir à la version d'avant
kubectl rollout undo deployment/nginx --to-revision=2  # revenir à une révision précise
```


C'est un **filet de sécurité** essentiel : un déploiement raté se corrige en quelques secondes, sans reconstruire quoi que ce soit.

5. Mettre en pause / reprendre

Pour grouper plusieurs changements en un seul déploiement :

```
kubectl rollout pause deployment/nginx    # geler les déploiements
# ... plusieurs modifications ...
kubectl rollout resume deployment/nginx    # tout appliquer d'un coup
```

6. Bonnes pratiques

- **Tagger précisément les images** (`nginx:1.28`), jamais `latest` en prod : on doit savoir exactement quelle version tourne et pouvoir y revenir.
- **Définir une readiness probe** (module 09) : sans elle, Kubernetes croit un Pod « prêt » dès qu'il démarre et peut router du trafic vers un nginx pas encore opérationnel.
- **Surveiller `rollout status`** : il échoue si la nouvelle version ne devient jamais prête, ce qui évite de remplacer tous les Pods par des Pods cassés.

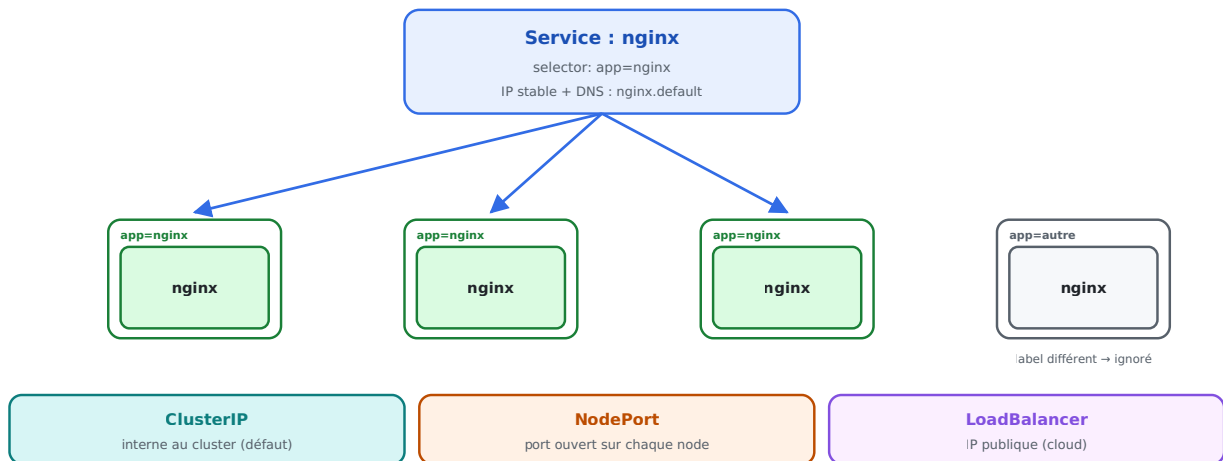
À retenir : le couple *Rolling Update* + *rollback* permet de déployer souvent et sans stress. Une mauvaise version ? `kubectl rollout undo` , et c'est réglé.

Les Services : exposer nginx

Les Pods sont **éphémères** et changent d'IP à chaque recreation. Comment leur parler de façon fiable ? Avec un **Service** : une adresse **stable** devant un groupe de Pods.

Le Service : une adresse stable devant des Pods éphémères

Le Service route le trafic vers les Pods qui portent le bon label (selector)



Le Service route le trafic vers les Pods qui portent le bon label (selector).

1. Le problème que le Service résout

```
kubectl get pods -o wide -l app=nginx
# nginx-aaa 10.244.1.7
# nginx-bbb 10.244.1.8 ← ces IP changent à chaque recreation !
```

On ne peut pas coder en dur l'IP d'un Pod. Le **Service** fournit :

- une **IP virtuelle stable** (ClusterIP) qui ne change jamais ;
- un **nom DNS** (`nginx.default.svc.cluster.local`) ;
- un **load-balancing** automatique entre tous les Pods correspondants.

2. Le mécanisme : labels & selector

Le Service ne connaît pas les Pods par leur nom : il les **sélectionne par label**.

```
apiVersion: v1
kind: Service
metadata:
  name: nginx
spec:
  selector:
    app: nginx          # cible TOUS les Pods portant label app=nginx
  ports:
    - port: 80          # port du Service
      targetPort: 80     # port du conteneur nginx
```

Le couplage labels ↔ selector est le ciment de Kubernetes. Tout Pod qui porte `app=nginx` est automatiquement intégré au Service — même créé après lui. Aucune configuration d'IP, jamais.

3. Les trois types de Service

Type	Portée	Usage typique
ClusterIP (défaut)	interne au cluster	communication entre services
NodePort	port ouvert sur chaque node	accès direct simple, tests
LoadBalancer	IP publique (cloud)	exposer un service sur Internet

ClusterIP — le défaut, interne

Accessible **uniquement depuis l'intérieur** du cluster. Les autres Pods joignent nginx simplement par son nom : `http://nginx`.

NodePort — un port sur chaque node

```
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080      # 30000–32767 ; accès via http://<IP-node>:30080
```

LoadBalancer — une IP publique (cloud)

```
spec:
  type: LoadBalancer
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
```

Sur un cloud (AWS, GCP, Azure), Kubernetes provisionne un vrai load-balancer avec une **IP publique**. En local (minikube), on utilise `minikube tunnel`.

4. Créer et tester un Service

```
kubectl expose deployment nginx --port=80 --target-port=80 # méthode rapide
# ou : kubectl apply -f nginx-service.yaml

kubectl get services          # voir l'IP et le type
kubectl describe service nginx # voir les "Endpoints" (IP des Pods ciblés)
```

Vérifier le load-balancing

```
# Depuis un Pod de test, plusieurs appels tombent sur des Pods différents
kubectl run tmp --rm -it --image=busybox -- wget -q0- http://nginx
```

5. La découverte de service par DNS

Kubernetes intègre un **DNS interne**. Tout Service est joignable par son nom :

Depuis...	On joint nginx par...
le même namespace	<code>nginx</code>
un autre namespace	<code>nginx.default</code>
nom complet	<code>nginx.default.svc.cluster.local</code>

C'est ainsi qu'un backend joint nginx, ou que nginx joint une API : **par nom**, jamais par IP.

6. Service vs Ingress

Un Service de type LoadBalancer = **une IP publique par service**, ce qui devient coûteux. Pour exposer **plusieurs** services HTTP derrière **une seule** entrée et router par URL, on utilise l'**Ingress** (module 08).

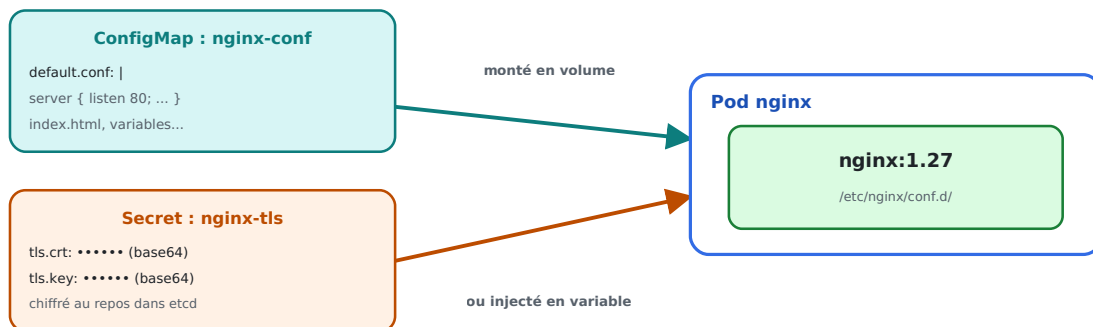
À retenir : Pods éphémères + Service stable = la combinaison qui rend une application joignable de façon fiable. Le Service suit automatiquement les Pods grâce aux labels.

ConfigMap & Secret : configurer nginx

Une bonne image est **générique** : la même image nginx doit servir en dev, en test et en prod, avec une **configuration différente** à chaque fois. Kubernetes sépare la config du conteneur grâce aux **ConfigMaps** et aux **Secrets**.

ConfigMap & Secret : séparer la config du conteneur

La même image nginx, configurée différemment selon l'environnement



On change la config sans rebuild l'image : il suffit de modifier le ConfigMap et de redémarrer les Pods.

La même image nginx, configurée différemment selon l'environnement — sans rebuild.

1. Le principe : externaliser la configuration

Objet	Pour	Stockage
ConfigMap	configuration non sensible (fichiers conf, variables, URL)	en clair
Secret	données sensibles (mots de passe, clés, certificats TLS)	encodé base64, chiffrable au repos

Pourquoi ? On ne rebuild **jamais** l'image pour changer un paramètre. On modifie le ConfigMap et on redémarre les Pods. La même image `nginx:1.27` tourne partout.

2. Un ConfigMap pour la config nginx

Créer le ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-conf
data:
  default.conf: |
    server {
      listen 80;
      location / {
        return 200 "Bonjour depuis Kubernetes\n";
      }
    }
  }
```

```
kubectl apply -f nginx-conf.yaml
kubectl get configmaps
kubectl describe configmap nginx-conf
```

Monter le ConfigMap comme fichier dans nginx

On injecte la conf dans `/etc/nginx/conf.d/` via un **volume** :

```
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      volumeMounts:
        - name: conf
          mountPath: /etc/nginx/conf.d      # nginx lit sa conf ici
  volumes:
    - name: conf
      configMap:
        name: nginx-conf
```

Le fichier `default.conf` du ConfigMap apparaît dans le conteneur — **sans rebuild**.

3. Injecter des valeurs en variables d'environnement

L'autre usage du ConfigMap : passer des **variables d'environnement**.

```
data:
  ENVIRONMENT: "production"
  WORKER_PROCESSES: "4"
```

```
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      envFrom:
        - configMapRef:
            name: nginx-conf          # toutes les clés deviennent des variables d'env
```

4. Les Secrets : pour les données sensibles

Un Secret ressemble à un ConfigMap, mais pour les données **confidentielles**. Les valeurs sont encodées en **base64**.

```
# Créer un Secret TLS depuis des fichiers certificat
kubectl create secret tls nginx-tls --cert=tls.crt --key=tls.key

# Ou un Secret générique
kubectl create secret generic db-pass --from-literal=password='S3cr3t!'
```

En YAML :

```
apiVersion: v1
kind: Secret
metadata:
  name: nginx-tls
type: kubernetes.io/tls
data:
  tls.crt: <contenu-base64>
  tls.key: <contenu-base64>
```

Utiliser un Secret dans le Pod

```
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      env:
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: db-pass
              key: password
      volumeMounts:
        - name: tls
          mountPath: /etc/nginx/tls          # certificats montés ici
  volumes:
    - name: tls
      secret:
        secretName: nginx-tls
```


5. ConfigMap vs Secret : que choisir ?

Donnée	Objet
Fichier <code>nginx.conf</code> , <code>index.html</code>	ConfigMap
URL d'API publique, niveau de log	ConfigMap
Mot de passe, token, clé API	Secret
Certificat / clé TLS	Secret (<code>type: kubernetes.io/tls</code>)

Attention : un Secret encodé en base64 **n'est pas chiffré** par défaut — base64 se décode trivialement. Pour une vraie protection : activer le *chiffrement au repos* d'etcd, restreindre les accès (RBAC), et ne **jamais committer** un Secret en clair dans Git.

6. Mettre à jour la configuration

```
kubectl edit configmap nginx-conf      # modifier
kubectl rollout restart deployment/nginx # redémarrer les Pods pour recharger
```

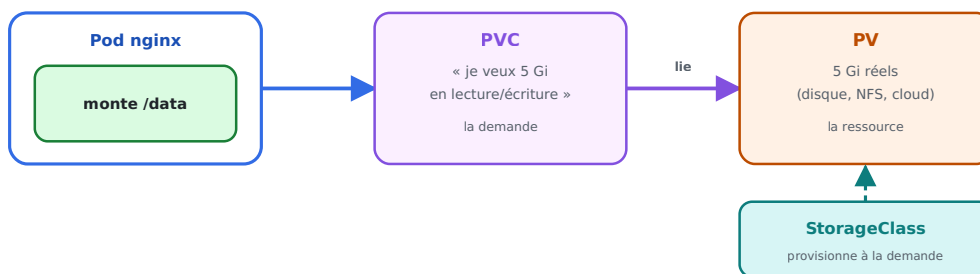
À retenir : image générique + config externalisée = portabilité totale. On déploie la **même** image nginx partout, et seuls le ConfigMap et le Secret changent.

Le stockage persistant

Les Pods sont **éphémères** : quand un Pod meurt, son disque disparaît avec lui. Pour conserver des données (logs, uploads, cache), il faut un **volume persistant** qui **survit** aux Pods.

Le stockage persistant : PVC > PV > StorageClass

Les Pods sont éphémères ; le volume persistant survit à leur suppression



Le Pod demande (PVC), Kubernetes fournit (PV) — automatiquement via la StorageClass.

Le Pod demande (PVC), Kubernetes fournit (PV) — automatiquement via la StorageClass.

1. Volume éphémère vs persistant

	Volume éphémère (emptyDir)	Volume persistant (PV/PVC)
Durée de vie	celle du Pod	indépendante du Pod
Pod supprimé	données perdues	données conservées
Usage	cache, scratch temporaire	données importantes

Un volume éphémère partagé entre conteneurs

```

spec:
  containers:
    - name: nginx
      image: nginx:1.27
      volumeMounts:
        - name: cache
          mountPath: /var/cache/nginx
  volumes:
    - name: cache
      emptyDir: {}          # créé avec le Pod, détruit avec lui
  
```

2. Les trois objets du stockage persistant

C'est le point qui déroute le plus. Trois objets se répartissent les rôles :

Objet	Qui le crée	Rôle
PV (PersistentVolume)	l'admin / la StorageClass	le stockage réel (disque, NFS, cloud)
PVC (PersistentVolumeClaim)	l'utilisateur	la demande : « je veux 5 Gi »
StorageClass	l'admin	provisionne un PV automatiquement à la demande

L'analogie : le **PVC** est un bon de commande (« je veux 5 Gi en lecture/écriture »), le **PV** est le disque livré, la **StorageClass** est le fournisseur qui fabrique le disque à la demande. Le Pod ne connaît que le **PVC**.

3. Réclamer du stockage : le PVC

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: nginx-data
spec:
  accessModes:
    - ReadWriteOnce          # monté en lecture/écriture par un seul node
  resources:
    requests:
      storage: 5Gi           # la taille demandée
      storageClassName: standard # quel "fournisseur" utiliser
```

```
kubectl apply -f nginx-pvc.yaml
kubectl get pvc          # statut : Bound (lié à un PV) ?
kubectl get pv           # le PV créé automatiquement
```

Les modes d'accès

Mode	Signification
<code>ReadWriteOnce</code> (RWO)	lecture/écriture par un seul node
<code>ReadOnlyMany</code> (ROX)	lecture seule par plusieurs nodes
<code>ReadWriteMany</code> (RWX)	lecture/écriture par plusieurs nodes (NFS...)

4. Monter le PVC dans le Pod nginx

```
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      volumeMounts:
        - name: data
          mountPath: /usr/share/nginx/html    # nginx sert ces fichiers
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: nginx-data                # référence au PVC
```

Désormais, ce qui est écrit dans `/usr/share/nginx/html` **survit** à la suppression et à la recréation du Pod.

5. La provision dynamique via StorageClass

Sans StorageClass, un admin devrait créer chaque PV à la main. La **StorageClass** automatise : quand un PVC arrive, elle **fabrique le PV** correspondant (un disque cloud, un volume NFS...).

```
kubectl get storageclass    # les classes disponibles (souvent une "default")
```

Si le PVC ne précise pas `storageClassName`, la **StorageClass par défaut** est utilisée.

6. Stockage et Deployments : le StatefulSet

Un Deployment convient pour des Pods **interchangeables** (comme nginx servant des fichiers statiques). Mais pour des applications à **identité stable et stockage individuel** (bases de données : PostgreSQL, MongoDB...), Kubernetes propose le **StatefulSet** :

- chaque Pod a un **nom stable** (`db-0`, `db-1` ...);
- chaque Pod garde **son propre** PVC, même après recréation.

nginx servant un site statique n'a généralement pas besoin de StatefulSet ; un Deployment + un PVC partagé (RWX) ou pas de persistance du tout suffit. Le StatefulSet est mentionné ici pour savoir **quand** s'en servir : les applications **avec état**.

7. Commandes utiles

```
kubectl get pvc,pv          # demandes et volumes
kubectl describe pvc nginx-data # statut, événements de binding
kubectl get storageclass     # fournisseurs disponibles
kubectl delete pvc nginx-data # libère le volume (selon la reclaim policy)
```

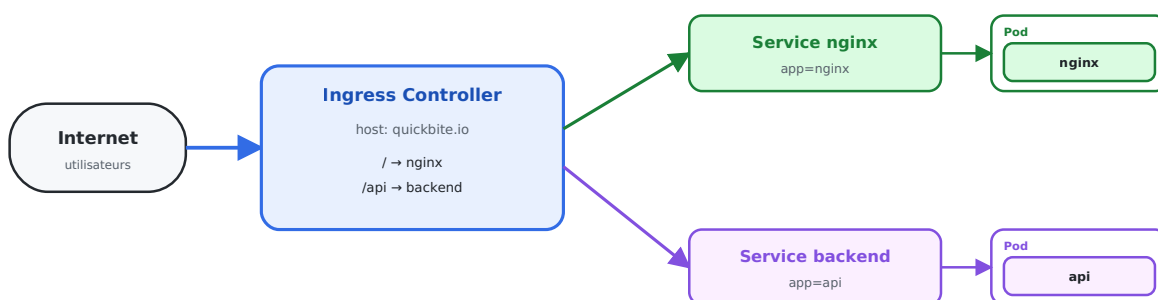
À retenir : le Pod demande (PVC), Kubernetes fournit (PV) via la StorageClass. C'est ce découplage qui rend le stockage portable d'un cluster à l'autre.

L'Ingress : router le trafic HTTP

Exposer chaque service avec un **LoadBalancer** coûte **une IP publique par service**. L'**Ingress** mutualise l'entrée : **une seule** porte HTTP/HTTPS qui route vers plusieurs services selon l'**URL** et le **domaine**.

L'Ingress : un routeur HTTP à l'entrée du cluster

Une seule IP publique route vers plusieurs Services selon l'URL



Sans Ingress, il faudrait un LoadBalancer (et une IP) par Service. L'Ingress mutualise l'entrée HTTP/HTTPS.

Une seule IP publique route vers plusieurs Services selon l'URL ou le domaine.

1. Le rôle de l'Ingress

L'Ingress est un **routeur HTTP/HTTPS de niveau 7** placé à l'entrée du cluster. Il sait :

- router selon le **chemin** : `/` → nginx, `/api` → backend ;
- router selon le **domaine** : `site-a.com` → service A, `site-b.com` → service B ;
- gérer le **TLS/HTTPS** de façon centralisée (un seul endroit pour les certificats).

Approche	Coût	Souplesse
Un LoadBalancer par service	une IP par service	aucune (pas de routage URL)
Un Ingress	une entrée mutualisée	routage par host et par chemin

2. Attention : l'Ingress a besoin d'un contrôleur

Point clé souvent oublié : l'objet **Ingress** n'est qu'un **ensemble de règles**. Pour qu'elles s'appliquent, il faut un **Ingress Controller** installé dans le cluster (le moteur qui lit les règles et route réellement le trafic).

Le plus courant est **ingress-nginx** (un nginx qui sert de routeur — encore lui !) :

```
# Sur minikube, une commande suffit
minikube addons enable ingress
kubectl get pods -n ingress-nginx      # le contrôleur tourne ?
```

3. Un Ingress qui expose nginx

On suppose un Service `nginx` (ClusterIP) déjà créé (module 05).

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: nginx
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
    - host: quickbite.local      # le domaine visé
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: nginx      # le Service ciblé
                port:
                  number: 80
```

```
kubectl apply -f nginx-ingress.yaml
kubectl get ingress      # voir l'adresse et les hosts
```

Tester en local

```
# Récupérer l'IP du contrôleur, puis simuler le domaine
echo "$(minikube ip) quickbite.local" | sudo tee -a /etc/hosts
curl http://quickbite.local      # → servi par nginx
```

4. Router plusieurs services

L'intérêt de l'Ingress : un **seul** point d'entrée vers **plusieurs** services.

Par chemin (path-based)

```
rules:
- host: quickbite.local
  http:
    paths:
      - path: /
        pathType: Prefix
        backend: { service: { name: nginx, port: { number: 80 } } }
      - path: /api
        pathType: Prefix
        backend: { service: { name: backend, port: { number: 8080 } } }
```

→ `quickbite.local/` va vers nginx, `quickbite.local/api` va vers le backend.

Par domaine (host-based)

```
rules:
- host: www.quickbite.local
  http: { paths: [ { path: /, pathType: Prefix,
    backend: { service: { name: nginx, port: { number: 80 } } } } ] }
- host: api.quickbite.local
  http: { paths: [ { path: /, pathType: Prefix,
    backend: { service: { name: backend, port: { number: 8080 } } } } ] }
```

5. Activer HTTPS (TLS)

On réutilise un **Secret TLS** (module 06) et on l'attache à l'Ingress :

```
spec:
  tls:
    - hosts:
        - quickbite.local
      secretName: nginx-tls      # le Secret type kubernetes.io/tls
  rules:
    - host: quickbite.local
      # ... mêmes règles
```

Le **certificat est géré au niveau de l'Ingress** : les Pods nginx, eux, restent en HTTP simple. (Avec *cert-manager*, on peut même obtenir des certificats Let's Encrypt automatiquement.)

6. Le chemin complet du trafic

```
Internet → Ingress Controller → Service (ClusterIP) → Pods nginx
          (routage par URL/host) (load-balancing)      (le conteneur)
```


Couche	Décide...
Ingress	vers quel Service envoyer (selon l'URL/host)
Service	vers quel Pod (load-balancing par label)
Pod	exécute nginx

À retenir : Ingress = routeur HTTP mutualisé à l'entrée du cluster. Il a besoin d'un *Ingress Controller*, il route par host/chemin et centralise le TLS. C'est la façon standard d'exposer des applications web sur Internet.

Pratique, santé & checklist

Ce dernier module rassemble les éléments qui rendent un déploiement **propre et fiable** : namespaces, sondes de santé, limites de ressources, un TP complet et un aide-mémoire.

1. Les Namespaces : cloisonner le cluster

Un **namespace** est une cloison logique pour organiser les objets (par équipe, par environnement...). Les noms doivent être uniques **dans** un namespace, pas entre eux.

```
kubectl create namespace dev
kubectl apply -f nginx-deployment.yaml -n dev    # déployer dans "dev"
kubectl get pods -n dev                        # lister dans "dev"
kubectl get pods -A                             # tous namespaces confondus
```

Namespace	Contenu
default	où atterrissent les objets sans namespace précisé
kube-system	les composants internes de Kubernetes
vos namespaces	dev , staging , prod , par équipe...

2. Les sondes de santé (probes)

Kubernetes vérifie l'état des conteneurs avec des **sondes**. Sans elles, il croit un Pod sain dès qu'il démarre — même si nginx n'est pas encore prêt.

Sonde	Question	Si échec
liveness	« le conteneur est-il vivant ? »	Kubernetes le redémarre
readiness	« peut-il recevoir du trafic ? »	il est retiré du Service
startup	« a-t-il fini de démarrer ? »	protège les démarrages lents

```
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      ports:
        - containerPort: 80
      livenessProbe:
        httpGet: { path: /, port: 80 }
        initialDelaySeconds: 5
        periodSeconds: 10
      readinessProbe:
        httpGet: { path: /, port: 80 }
        initialDelaySeconds: 3
        periodSeconds: 5
```

La readiness probe est cruciale pendant un rolling update : Kubernetes n'envoie du trafic vers un nouveau Pod nginx que lorsque la sonde le déclare **prêt**. C'est ce qui garantit le « zéro coupure » du module 04.

3. Requests & limits : maîtriser les ressources

On déclare ce qu'un conteneur **demande** (`requests` , utilisé par le scheduler pour le placer) et son **plafond** (`limits` , au-delà duquel il est bridé ou tué).

```
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      resources:
        requests:           # le minimum garanti (pour le placement)
          cpu: "100m"       # 0,1 cœur
          memory: "64Mi"
        limits:             # le maximum autorisé
          cpu: "500m"
          memory: "128Mi"
```

Champ	Rôle
<code>requests</code>	réservation garantie ; sert au scheduler
<code>limits</code>	plafond ; dépassement mémoire → Pod tué (OOMKilled)

Bonne pratique : toujours définir `requests / limits` en production, sinon un Pod peut monopoliser un node et affamer les autres.

4. TP complet : nginx de A à Z

Déployons nginx avec tout ce qu'on a vu, dans le namespace `demo`.

```
# 1) Namespace
kubectl create namespace demo

# 2) Config (ConfigMap)
kubectl create configmap nginx-conf --from-file=default.conf -n demo

# 3) Deployment : 3 réplicas
kubectl apply -f nginx-deployment.yaml -n demo
kubectl get pods -n demo                # 3 Pods Running

# 4) Service : adresse stable
kubectl expose deployment nginx --port=80 -n demo
kubectl get svc -n demo

# 5) Exposition externe (Ingress)
kubectl apply -f nginx-ingress.yaml -n demo

# 6) Mise à jour sans coupure
kubectl set image deployment/nginx nginx=nginx:1.28 -n demo
kubectl rollout status deployment/nginx -n demo

# 7) Oups, on revient en arrière
kubectl rollout undo deployment/nginx -n demo

# 8) Scaling
kubectl scale deployment nginx --replicas=5 -n demo

# 9) Nettoyage complet
kubectl delete namespace demo          # supprime TOUT le namespace
```

5. Aide-mémoire kubectl

```
# Observer
kubectl get pods|svc|deploy|ingress|pvc      # lister un type d'objet
kubectl get all -n <ns>                      # tout dans un namespace
kubectl get pods -o wide                     # avec IP et node
kubectl get pods -w                          # mode "watch" (temps réel)

# Comprendre / déboguer
kubectl describe pod <nom>                   # événements détaillés
kubectl logs <pod> [-f]                     # logs (-f = suivre)
kubectl exec -it <pod> -- sh                 # shell dans le conteneur
kubectl get events --sort-by=.lastTimestamp  # derniers événements

# Agir
kubectl apply -f fichier.yaml                # créer/mettre à jour (déclaratif)
kubectl delete -f fichier.yaml              # supprimer
kubectl scale deploy/<nom> --replicas=N      # scaler
kubectl rollout undo deploy/<nom>           # rollback
kubectl explain <objet>.spec                # doc d'un champ YAML
```

6. Erreurs fréquentes & solutions

Symptôme	Cause probable	Solution
ImagePullBackOff	image introuvable / mal taguée	vérifier le nom et le tag de l'image
CrashLoopBackOff	l'app plante au démarrage	<code>kubectl logs <pod></code>
Pending qui dure	pas assez de ressources / pas de PV	<code>kubectl describe pod</code> (events)
Service ne répond pas	selector $\neq$ labels des Pods	vérifier les Endpoints
PVC en Pending	pas de StorageClass / PV	<code>kubectl get sc</code> , <code>describe pvc</code>
Ingress 404	pas de contrôleur / mauvais host	installer ingress-nginx, vérifier <code>host:</code>

Méthode universelle : `kubectl get pods` → repérer le statut anormal → `kubectl describe pod <nom>` (section *Events*) → `kubectl logs <nom>`. 90 % des pannes se diagnostiquent ainsi.

7. Checklist d'un déploiement propre

- [] Application déployée via un **Deployment** (jamais un Pod nu).
- [] **Labels** cohérents entre Deployment, Pods et Service (`app=nginx`).
- [] **Service** pour une adresse stable ; **Ingress** pour l'accès HTTP externe.
- [] **liveness** et **readiness** probes définies.
- [] **requests/limits** CPU et mémoire fixées.

- [] Config externalisée en **ConfigMap**, secrets en **Secret** (jamais dans l'image).
 - [] **PVC** pour les données à conserver ; éphémère sinon.
 - [] Images **taguées précisément** (pas `latest` en prod).
 - [] Objets rangés dans un **namespace** dédié.
 - [] Stratégie de **rolling update** + rollback validée.
-

Conclusion

Avec une seule image — **nginx** — vous avez parcouru tous les concepts essentiels de Kubernetes : **Pods**, **Deployments**, **rolling updates**, **Services**, **ConfigMaps** et **Secrets**, **volumes persistants** et **Ingress**, plus les sondes de santé, les ressources et les namespaces.

Le fil conducteur reste toujours le même : on **déclare un état désiré** en YAML, et Kubernetes **converge** vers cet état, en réparant et en répliquant tout seul. C'est ce qui transforme des conteneurs fragiles en applications **résilientes, scalables et auto-réparées**.

Pour aller plus loin : Helm (packager ses manifestes), RBAC (droits d'accès), les NetworkPolicies (cloisonnement réseau), le monitoring (Prometheus/Grafana), les Operators, et le GitOps (Argo CD / Flux) pour déployer sur Kubernetes... depuis Git.